# Softwareentwicklung mit Java 2



## 1. Klassen

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# Klassen

## Definition von Bausteinen mit Daten und Operationen

- Datenkomponenten (***Variablen***, ***Felder***)
- Methoden

## Schablone für Objekterzeugung

- Erzeugen mit **new**

```
new Timer();
```

## Repräsentieren Typen

- für Deklarationen von Variablen, Parameter etc.

```
Timer timer;
```

```
public class Timer {

  int hrs;
  int min;
  int sec;

  void clear() {
    hrs = 0;
    min = 0;
    sec = 0;
  }

  void add(int s) {
    sec += s;
    if (sec > 59) {
       min += sec / 60;
       sec = sec % 60;
    }
    if (min > 59) {
      hrs += min / 60;
      min = min % 60; }
  }
}
```

Klassenkopf mit Name

Daten (Felder)

Methoden

© H. Prähofer, 2026

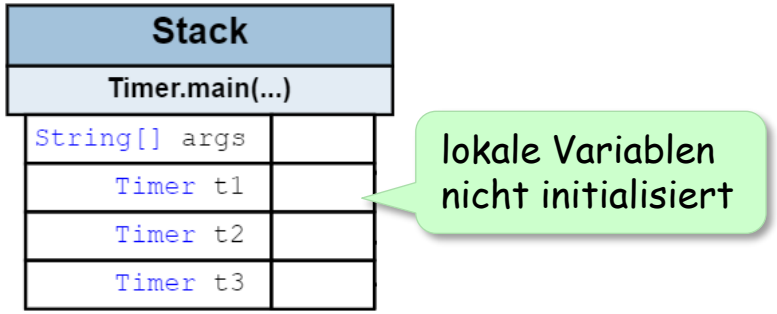
# Variablendeklaration und Objekterzeugung

## Deklaration von Variablen → Klasse als Typ

- lokale Variablen/Parameter werden am Stack angelegt

Typ der Variablen **t1**, **t2**, **t3**

```
public static void main(String[] args) {
  Timer t1, t2, t3;
```

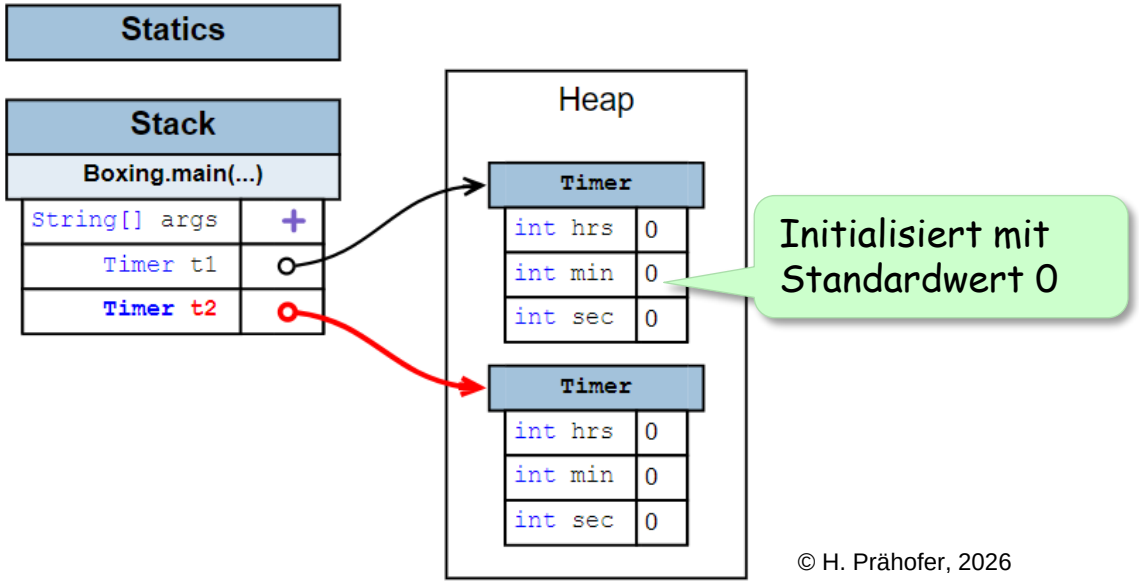



## Erzeugung von Objekten → Klasse als Schablone für Objekte

- Objekte werden mit **new** am Heap angelegt!!
- Resultat ist eine Referenz auf Heap-Objekt
- Zugriff auf Heap-Objekte immer über Referenz

Schablone für **Timer**-Objekt

```
  t1 = new Timer();
  t2 = new Timer();
```




© H. Prähofer, 2026

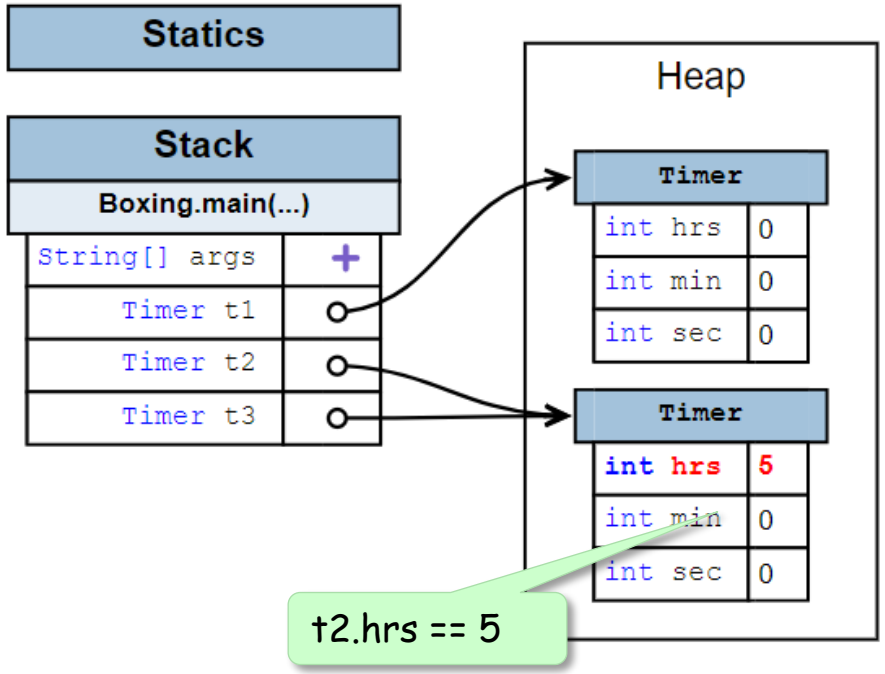
# Referenzen und Referenzzuweisungen

## Referenzzuweisungen

- Bei Zuweisungen werden Referenzen zugewiesen

```
t3 = t2;
```

Referenzzuweisung




## Seiteneffekte

- Mit Referenzzuweisungen sind Seiteneffekte möglich

  ➔ d.h. bei einer Änderung über eine Variable, wird auch der „Wert“ der anderen Variablen verändert.

```
t3.hrs = 5;
```

```
System.out.println("t2.hrs: " + t2.hrs);
```

```
t2.hrs: 5
```

Beachte:

- Seiteneffekte sind potentielle Fehlerquellen
- Grund: sind nicht direkt ersichtlich und damit oft schwer zu verstehen
- Seiteneffekte sollen möglichst vermieden werden

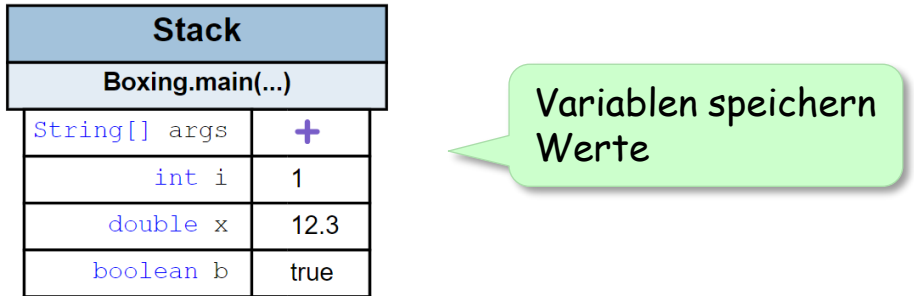
© H. Prähofer, 2026

# Wertesemantik vs. Referenzsemantik

## Wertesemantik

- Basisdatentypen
  - **int, long, byte, short**
  - **double, float**
  - **char, boolean**
- Werte werden direkt gespeichert
- Zuweisungen kopieren Werte
  - ➔ **keine Seiteneffekte**

```
public static void main(String[] args) {
  int i = 1;
  double x = 12.3;
  boolean b = true;
  int i2 = i;
}
```



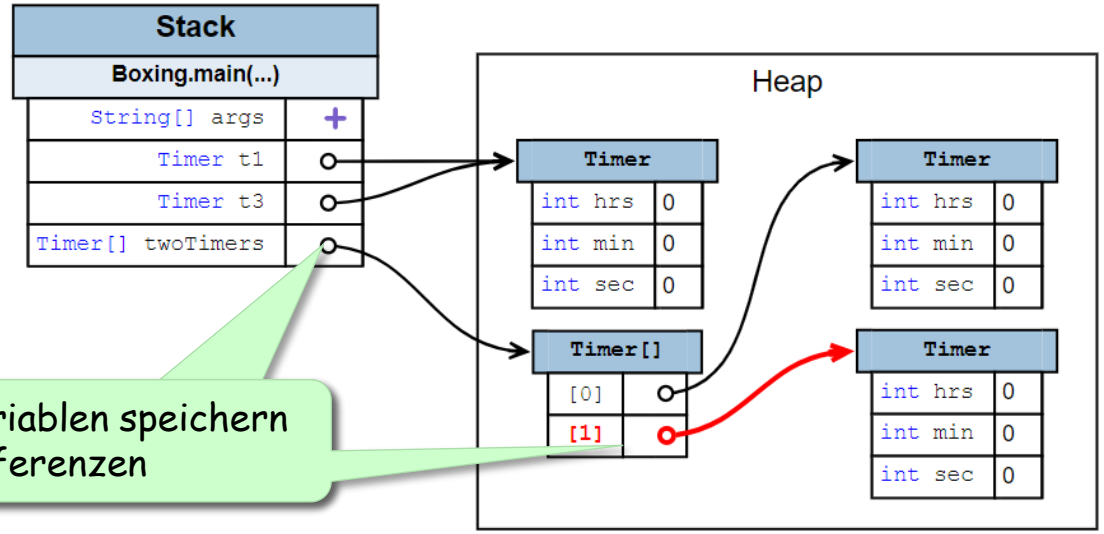

Variablen speichern Werte

## Referenzsemantik

- Objekte von Klassen und Arrays werden am Heap angelegt
- Variablen speichern Referenzen auf Heap-Objekte
- Zuweisungen kopieren Referenzen
  - ➔ **Seiteneffekte**

```
public static void main(String[] args) {
  Timer t1 = new Timer();
  Timer t3 = t1;

  Timer[] twoTimers = new Timer[2];
  twoTimers[0] = new Timer();
  twoTimers[1] = new Timer();
}
```




Variablen speichern Referenzen

© H. Prähofer, 2026

# Wrapper-Klassen und Boxing / Unboxing

## Wrapper-Klassen für Basisdatentypen

- zu den Basisdatentypen gibt es Wrapper-Klassen
- werden eingesetzt, wenn man für Wert eines Basisdatentyps ein Heap-Objekt benötigt

| Wertetypen | Wrapper-Klassen |
|---|---|
| boolean | Boolean |
| int | Integer |
| long | Long |
| byte | Byte |
| short | Short |
| char | Character |
| double | Double |
| float | Float |

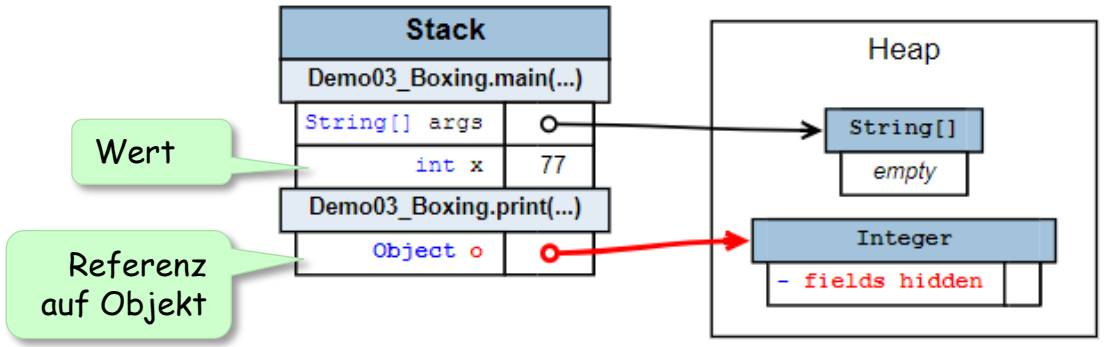
Beachte Großbuchstaben

z.B. wenn man eine Methode mit einem **Object**-Parameter mit einem **int**-Wert aufrufen will

```
static void print(Object o) {
  System.out.println(o.toString());
}
```

Parameter Object

```
int x = 77;
print(Integer.valueOf(x));
```




© H. Prähofer, 2026

# Boxing und Unboxing

- **Boxing**: Automatisches Konvertieren von Werten eines Basisdatentyps in Heap-Objekt

```
int x = 1;
Integer xObj = x;
```

```
static void print(Object o) {
  System.out.println(o.toString());
}
```

```
int x = 77;
print(x);
```

automatisches Boxing

- **Unboxing**: Automatisches Konvertieren von Wrapper-Objekt in Wert des Basisdatentyps

```
int y = xObj;
```

© H. Prähofer, 2026

# null-Referenz

- **null** bedeutet **Referenz auf kein Objekt**
- **null** kann jeder Variablen eines Referenztyps als Wert zugewiesen werden

```
Person p = null;
```

- Typprüfung erlaubt Zugriffe aufgrund des Referenztyps – unabhängig vom tatsächlichen Wert
  ➔ damit auch Zugriff erlaubt, wenn der Wert **null** ist

!!

- aber Zugriffe über eine **null**-Referenz führen zu einer **NullPointerException** zur Laufzeit

```
System.out.format("%s", p.firstName());
```

NullPointerException

Tony Hoare, der Erfinder der null-Referenz:

*„I call it my billion-dollar mistake. It was the invention of the null reference in 1965."*

```
public class Person {

  private String firstName;
  private String midName;
  private String lastName;

  public Person(String firstName,
                String midName,
                String lastName) {
    this.firstName = firstName;
    this.midName = midName;
    this.lastName = lastName;
  }

  public String firstName() {
    return firstName;
  }

  public String midName() {
    return midName;
  }

  public String lastName() {
    return lastName;
  }
}
```

© H. Prähofer, 2026

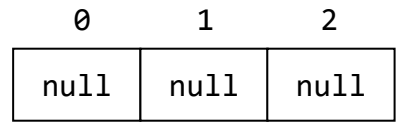
# NULL HAT MEHRERE BEDEUTUNGEN/VERWENDUNGEN

## Zustand einer Referenz

- Person ist unbekannt
- Person existiert nicht
- noch kein Objekt zugewiesen
  ➔ z. B. Felder und Array-Elemente werden standardmäßig mit null initialisiert

```
Person p = null;
```

```
Person[] persons = new Person[3];
Person p = persons[0];
```




## Verwendung in einer API

- Sind **null**-Werte erlaubt?

```
Person p = new Person(null, null, null);
int n = p.firstName().length();
```

NullPointerException

- **null** für optionalen Wert

```
Person p = new Person("Heinz", null, "Burger");
```

```
Heinz null Burger
```

- **null** als Wert für Zugriff/Suche war erfolglos

```
Person p = findPerson("Burger");
```

**null** für nicht gefunden

© H. Prähofer, 2026

# Umgang mit null-Werten: Prüfen und Ersetzen

- Prüfen: expliziter Vergleich mit **null**

```
if (p != null) {
  System.out.format("%s %s %s%n",
                    p.firstName(), p.midName(), p.lastName());
}
```

- Zurückweisen: **Objects.requireNonNull**
  - wirft bei **null** eine **NullPointerException**

```
public Person(String firstName, String midName, String lastName) {
  Objects.requireNonNull(firstName, "firstName must not be null");
  Objects.requireNonNull(lastName, "lastName must not be null");
  this.firstName = firstName;
  this.midName = midName;
  this.lastName = lastName;
}
```

- Ersetzen: **Objects.requireNonNullElse**
  - alternativer Wert wenn **null**

```
public class Person {

  @Override
  public String toString() {
    return "%s %s %s".formatted(firstName,
        Objects.requireNonNullElse(midName, ""),
        lastName);
  }
}
```

© H. Prähofer, 2026

# Umgang mit null-Werten: null-sichere Operationen

- Null-sicherer Wertevergleich
  - **equals** wirft **NullPointerException**, wenn Empfängerobjekt **null**
  - Sicherer Verglich mit **Objects.equals**

**NullPointerException**, wenn **p == null**

```
if (p.equals(p2)) { … }
```

```
if (Objects.equals(p, p2)) { … }
```

```
public static boolean equals(Object a, Object b) {
    return (a == b) || (a != null && a.equals(b));
}
```

- Null-sichere String-Konvertierung:
  - **toString** wirft **NullPointerException**, wenn Empfängerobjekt **null**
  - Mit **String.valueOf** wird bei Wert **null** **"null"** ausgegeben

**NullPointerException**, wenn **p == null**

```
System.out.println(p.toString());
```

```
System.out.println(String.valueOf(p));
```

```
null
```

```
public static String valueOf(Object obj) {
    return (obj == null) ? "null" : obj.toString();
}
```

- Später:
  - Verwendung von **Optional** ➔ siehe *Kapitel* **4.** ***Collections und Optional***
  - Annotationen wie **@NonNull** und **@Nullable** ➔ siehe Kapitel **8. Reflection und Annotationen**

© H. Prähofer, 2026

# Lernziele überprüfen

- Was sind Basisdatentypen und was sind Referenztypen?
- Wie unterscheiden sich Basisdatentypen und Referenztypen?
- Was versteht man unter Referenzsemantik?
- Was passiert bei einer Referenzzuweisung?
- Wie entstehen Seiteneffekte bei Objekten und warum sind Seiteneffekte schlecht?
- Was sind Wrapper-Klassen und wozu dienen sie?
- Was passiert beim Boxing und Unboxing?
- Welche Bedeutung hat der Wert null?
- Welche Möglichkeiten gibt es, mit null-Werten umzugehen?

© H. Prähofer, 2026

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# Bestandteile von Klassen

```
class Klassenname extends Superklasse
implements Interface, ... {

    Klassen (static)
        statische Variablen (statische Felder)
        statischer Initialisierungsblock
        statische Methoden

    Objekt
        Variablen (Felder)
        Konstruktoren
        Methoden
}
```

- Klassenkopf mit **Superklasse** und implementierten **Interfaces**

- **statische** Elemente gehören **zur Klasse**
  - ➔ Felder existieren **einmal pro Klasse**
  - ➔ statischer Initialisierungsblock zum **Intialisieren der statischen Felder**
  - ➔ Methodenaufrufe für **Klasse**

- **nicht-statische** Elemente gehören **zu den Objekten**
  - ➔ Felder existieren **einmal pro Objekt**
  - ➔ Konstruktor für **Erzeugen von Objekten**
  - ➔ Methodenaufrufe für **Objekt**

© H. Prähofer, 2026

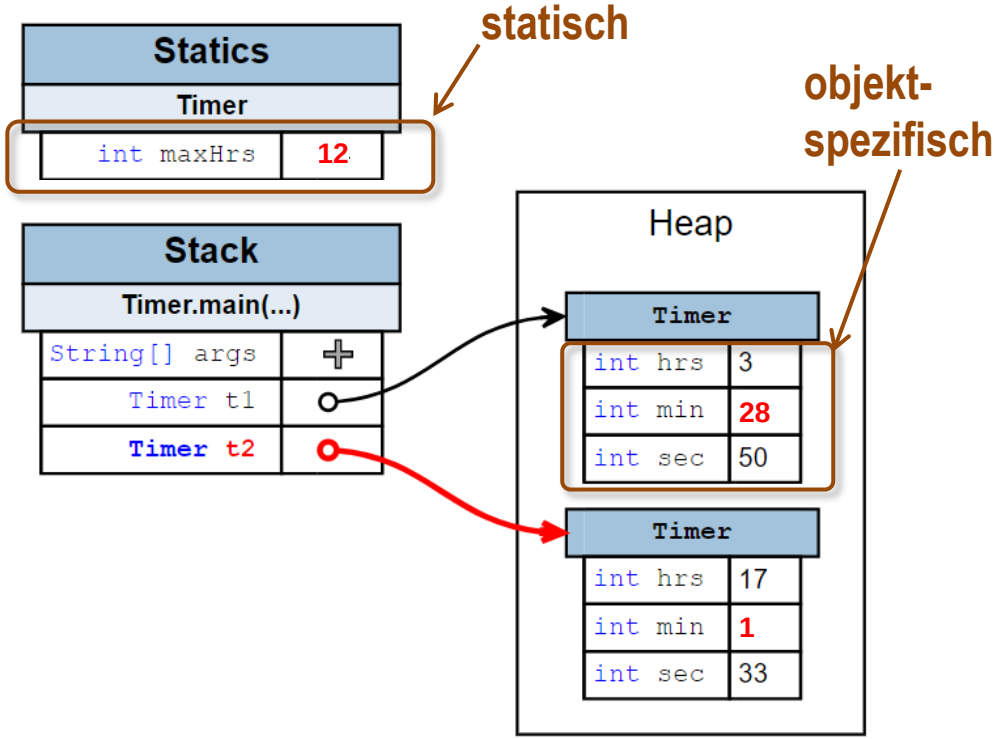
# Felder: Statische vs. Objektvariablen

```
class Timer {
  static int maxHrs = 24;

  int hrs;
  int min;
  int sec;
}
```

**Statische Variable (Klassenfeld)**
nur einmal pro Klasse vorhanden

**Objektvariablen (Felder)**
in jedem Objekt vorhanden




Statische Variablen: Zugriff über Klassennamen

```
Timer.maxHrs = 12;
```

```
Timer t1 = new Timer(3, 27, 50);
Timer t2 = new Timer(17, 0, 33);
```

Objektvariablen: Zugriff über Objektreferenz

```
t1.min = t1.min + 1;
t2.min = 1;
```

© H. Prähofer, 2026

# Methoden: Statische vs. Objektmethoden

```
public class Timer {
  static int maxHrs = 24;
  ...
  static void switchMaxHrs() {
    maxHrs = 36 - maxHrs;
  }

  void clear() {...}

  void add(int s) {...}
  ...
}
```

**Statische Methode (Klassenmethode)**

- wird auf die Klasse selbst angewendet
- Aufruf über Klassennamen

```
Timer.switchMaxHrs();
```

**Objektmethoden**

- werden auf Objekte der Klasse angewendet
- Aufruf über Objektreferenzen

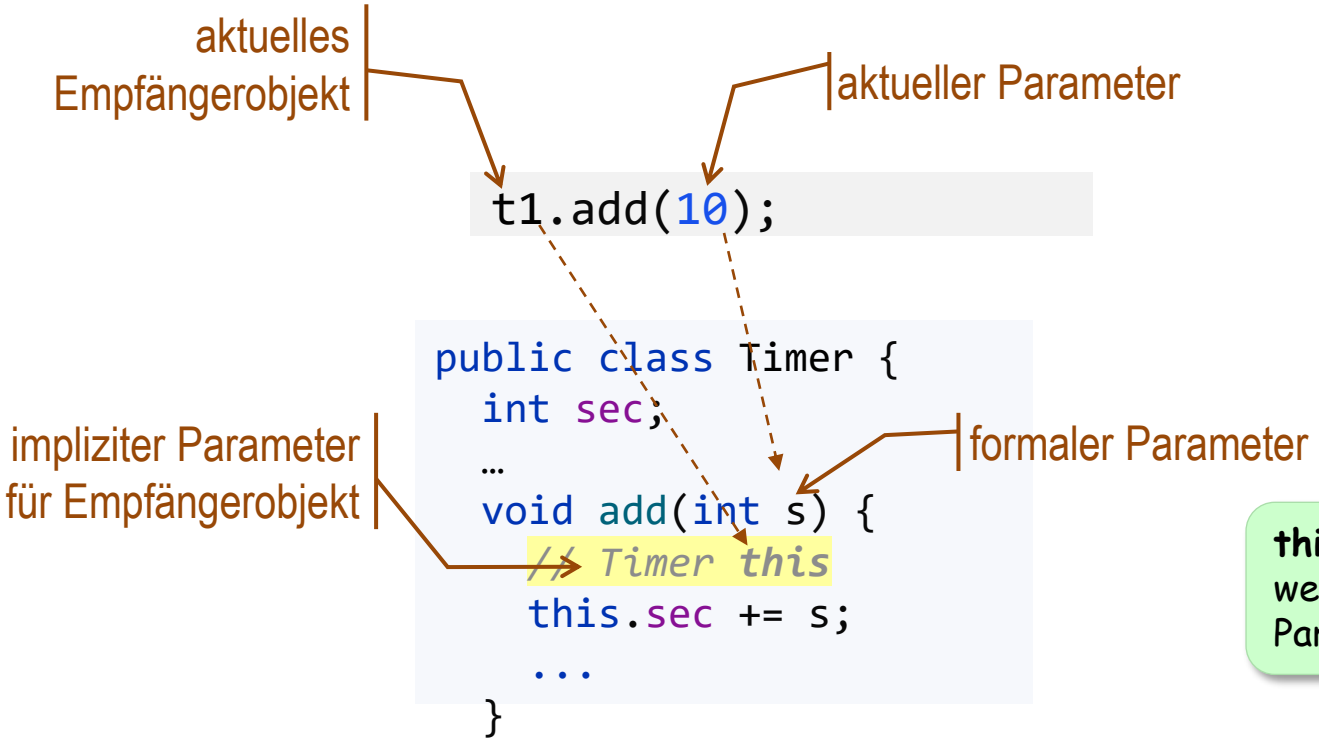
```
t1.add(10);
```

```
t2.add(5);
```

© H. Prähofer, 2026

# Objektmethoden: Aufruf

- **Objektmethoden** werden für **Empfängerobjekt** aufgerufen
- **this** ist impliziter Parameter für Empfängerobjekt
  - Typ gleich der Klasse
  - Wert Referenz auf Empfängerobjekt




- **this** ist optional

```
public class Timer {
  int sec;
  …
  void add(int s) {
    sec += s;
    ...
  }
```

**this** nicht notwendig

- nur notwendig
  - wenn Zugriff nicht eindeutig

```
public class Timer {
  int sec;
  …
  void add(int sec) {
    this.sec += sec;
    ...
  }
```

**this** notwendig, weil Konflikt mit Parameter **sec**

  - wenn Referenz benötigt

© H. Prähofer, 2026

# Methoden: Methodensignatur

***Sichtbarkeit* [static] [abstract | final] *Typ Name* (*Parameterliste*) [ throws *geworfene Exceptions* ]**

## Beispiele:

```
private abstract void write(String text) throws IOException
```

```
public  final  int  readInt()  throws NumberFormatException, IOException
```

```
public String readString(String defaultValue)
```

## Parameterliste

- keine Parameter

```
int readInt()
```

- mehrere Parameter

```
void writeElems(List<String> elems, String separator)
```

- *vararg*-Parameter(als letzter Parameter)

nächste Folie

© H. Prähofer, 2026

# Methoden: Vararg-Parameter

## Letzter Parameter kann Vararg-Parameter sein

- mit Typ und drei Punkten deklariert: ***Type…***

```
void writeNumbers(String separator, int... numbers) {…}
```

- beim Aufruf können beliebig viele Werte angegeben werden

auch keine Werte erlaubt

beliebig viele Argumentwerte

```
writeNumbers(", ", 1, 2, 3, 4);
```

- Innerhalb Methode sind Werte in Array gespeichert

```
void writeNumbers(String separator, int... numbers) {
  for (int i = 0; i < numbers.length; i++) {
    System.out.print(numbers[i]);
    if (i < numbers.length-1) {
      System.out.print(separator);
    }
  }
```

int[] numbers

© H. Prähofer, 2026

# Methoden: Überladen von Methoden

## Überladen von Methoden durch unterschiedliche Parameterlisten

**Typen** der Parameter und **Anzahl** der Parameter sind wesentlich

Auswahl erfolgt auf Basis der **statischen Typen** der **aktuellen Parameter**

```
void write(String text) {...}
```
← `write("This is a string");`

```
void write(String text, int n) {...}
```
← `write("This is a string", 3);`

```
void write(int x) {...}
```
← `write(1);`

```
void write(Object o) {...}
```
← 
```
Object obj = "This is a string";
write(obj);
```

Auswahl auf Basis der **statischen** Typen

Anmerkung: Unterschiedliche Rückgabewerte und Exceptions führen zu keinem Überladen von Methoden und sind daher nicht erlaubt

```
void write(String text) {...}
```

~~`boolean write(String text) { ... }`~~

```
int readInt() {...}
```

~~`int readInt() throws IOException`~~
~~`int readInt() throws FormatException`~~

© H. Prähofer, 2026

# Statischer Initialisierungsblock

```
public class Timer {
  static int maxHrs = 24;

  static {
    // lies maxHrs aus Datei
    maxHrs = readFromFile("Timer.init");
    ...
  }
  ...
}
```

**Statischer Initialisierungsblock**
zum Initialisieren der Klassenvariablen

## Statischer Initialisierungsblock

- mit Schlüsselwort **static** gekennzeichnet
- hat keinen Namen und keine Parameterliste
- wird automatisch aufgerufen, bevor die Klasse zum ersten Mal benutzt wird
- Jede Klasse hat höchstens 1 statischen Initialisierungsblock
- Anwendung bei:
  - Initialisieren von statischen Feldern
  - Aktionen, die beim Laden der Klasse stattfinden sollen

© H. Prähofer, 2026

# Konstruktoren

## Spezielle Methoden zum Initialisieren von Objekten

- mit gleichem Namen wie die Klasse
- zum Setzen der Felder
- Konstruktoren können überladen werden
- überladene Konstruktoren können mit **this** einen anderen Konstruktor aufrufen

Häufig mehrere überladene Konstruktoren

Best Practice!

```
public class Timer {
  ...
  int hrs, min, sec;

  Timer(int h, int m, int s) {
    hrs = h; min = m; sec = s;
  }

  Timer() {
    this(0, 0, 0);
  }
  void clear() {...}
  void add(int s) {...}
}
```

Aufruf des überladenen Konstruktors

```
Timer t1 = new Timer(3, 25, 10);
```

```
Timer t2 = new Timer();
```

© H. Prähofer, 2026

# Standardkonstruktor

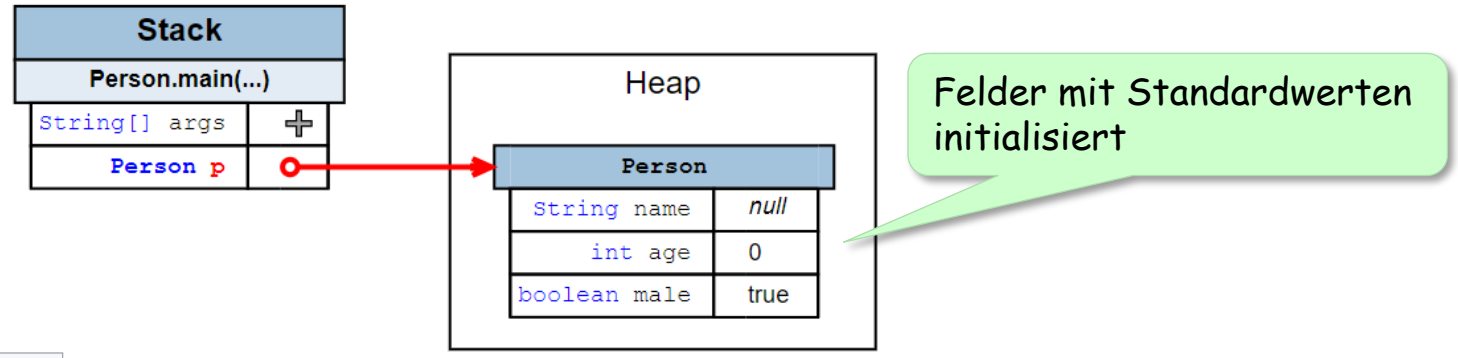
Wenn kein Konstruktor definiert, dann erzeugt Compiler einen **parameterlosen Standardkonstruktor**

- Die Felder sind dann mit Standardwerten (**null**, **0**, **false**, **'\0'**) initialisiert

```
public class Person {
  String name;
  int age;
  boolean male = true;
}
```

```
Person p = new Person();
```

← Ruft Standardkonstruktor auf




Aber: Kein Standardkonstruktor wird erzeugt, wenn ein anderer Konstruktor definiert wird

```
public class Person {
  String name;
  int age;
  boolean male = true;

  Person(String name, int age, boolean male) {
    this.name = name;
    this.age = age;
    this.male = male
  }
}
```

```
Person p = new Person();
```

✗ Fehler! Compiler erzeugt keinen Standardkonstruktor

© H. Prähofer, 2026

# Lernziele überprüfen

- Was sind die Bestandteile von Klassen?
- **this**: Was ist **this**? Wie wird **this** erzeugt und gesetzt? Was ist der Typ von **this**?
- Was sind Vararg-Parameter und wie werden sie deklariert?
- Was bedeutet Überladen von Methoden?
- Wie funktioniert Überladen von Methoden?
- Unterscheiden Sie Überschreiben und Überladen von Methoden?
- Wozu dienen Konstruktoren.
- Was bedeutet ein Aufruf mit **this** bei einem Konstruktor und wann soll dieser verwendet werden?
- Was ist der Standardkonstruktor und wann wird dieser automatisch zur Verfügung gestellt?

© H. Prähofer, 2026

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# Verzeichnisstruktur und Packages

- Java-Dateien werden in Dateiverzeichnissen organisiert




Dateiverzeichnis **hello**

Java-Datei in Verzeichnis **hello**

- Dateiverzeichnisse entsprechen Packages in Java-Programmen
  - Package **muss** am Anfang der Java-Datei angegeben werden

**Datei**: hello\HelloWorld.java

```
package hello;

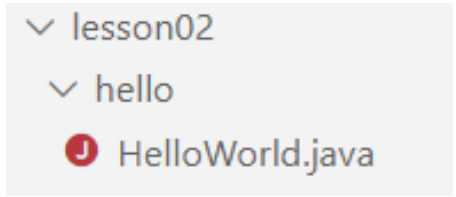
public class HelloWorld {  ... }
```

Schlüsselwort **package**
gefolgt von Packagenamen

© H. Prähofer, 2026

# Verzeichnisstruktur und Packages

- Hierarchische Verzeichnisse und Packages
  - Dateiverzeichnisse mit Unterverzeichnissen
  - Package mit Unterpackages
    ➔ mit Punkt getrennt




**Datei**: lesson02\hello\HelloWorld.java

Punkt!

```
package lesson02.hello;

public class HelloWorld {
   ...
}
```

© H. Prähofer, 2026

# Vollqualifizierter Klassenname

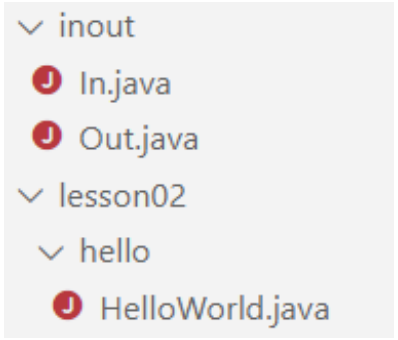
## Vollqualifizierter Klassenname = Package plus Klassenname getrennt mit Punkt

➔ erlaubt Identifikation einer Klasse innerhalb des Gesamtsystems

```
lesson02.hello.HelloWorld
```

- Verwendung von Klassen aus anderen Packages über vollqualifizierten Namen

```
package lesson02.hello;
public class HelloWorld {
  public static void main(String[] args) {
    String name = inout.In.readWord();
    inout.Out.println("Hello " + name);
  }
}
```




```
package inout;

public class Out { … }
```

```
package inout;

public class In { … }
```

- Ermöglicht Klassen mit gleichen Namen in unterschiedlichen Packages
  - z.B. **Date** in **java.util** und **Date** in **java.sql**

```
package java.util;

public class Date { … }
```

```
package java.sql;

public class Date { … }
```

```
public class Main {
  public static void main(String[] args) {
    java.util.Date date;
    java.sql.Date dbDate;
    …
```

Unterscheidung der Date-Klassen über vollqualifizierten Namen

© H. Prähofer, 2026

# Imports

- Imports von Klassen statt Verwendung des vollqualifizierten Namens

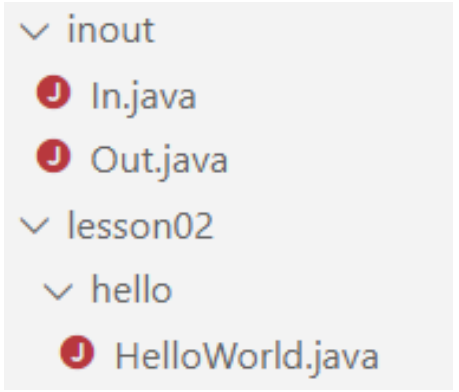
```
package lesson02.hello;
import inout.Out;

public class HelloWorld {

  public static void main(String[] args) {
    Out.println("Hello World!");
  }
}
```

Importieren von **Out** aus Package **inout**

Verwenden von **Out ohne Package**




- Import mit * ➔ Importieren aller public Klassen im Package

```
package lesson02.hello;
import inout.*;

public class HelloWorld {

  public static void main(String[] args) {
    String name = In.readWord();
    Out.println("Hello " + name);
  }
}
```

Importieren aller Klassen von **inout**

Verwenden von **In und Out ohne Package**

© H. Prähofer, 2026

# Statische Imports

## statische Imports importieren statische Members von Klassen

- mit **import static**
- mit vollqualifiziertem ***Klassenamen*** . ***statischem Member*** oder ***

```
import static Classname.staticMember
```

```
import static Classname.*
```

```
package lesson02.hello;

import static inout.In.readWord;
import static inout.Out.*;

public class HelloWorld {

  public static void main(String[] args) {
    String name = readWord();
    println("Hello " + name);
  }
}
```

Importiert statische Methode **readWord** von Klasse **inout.In**

Importiert alle statische Methoden von Klasse **inout.Out**

Aufruf von **inout.In.readWord** ohne Klassenname

Aufruf von **inout.Out.println** ohne Klassenname

© H. Prähofer, 2026

# Wichtige Packages aus JDK

| | |
|---|---|
| `java.lang` | Fundamentale Typen, z.B. **Object** und **String** |
| `java.lang.reflect` | Reflection API: Typen zur Darstellung von Klassen |
| `java.math` | Rechnen mit Zahlen beliebiger Genauigkeit |
| `java.text` | Utilities zur Textverarbeitung |
| `java.time` | Neue Klassen für Datum und Uhrzeit |
| `java.io` | Elementare Ein/Ausgabe |
| `java.nio` | Neue Ein/Ausgabe mit Files und Channels |
| `java.net` | Netzwerkzugriff über Sockets |
| `java.util` | Unterschiedliche Utility-Klassen inkl. Collections |
| `java.util.concurrent` | Utility-Klassen für Multithreading-Anwendungen |
| `java.util.function` | Funktionale Interfaces |
| `java.util.regex` | Regular Expressions |
| `java.util.stream` | Funktionale Streams |

Aus **java.lang** kein Import notwendig!

© H. Prähofer, 2026

# Lernziele überprüfen

- Was sind Packages und wozu werden sie verwendet?
- Wie werden Packages in Java deklariert und wie hängen sie mit der Verzeichnisstruktur zusammen?
- Was ist der vollqualifizierte Klassenname und wozu wird er benötigt?
- Wozu dienen Imports und wie werden Klassen aus anderen Packages importiert?
- Was bedeutet ein *-Import und welche Klassen werden damit importiert?
- Was sind statische Imports und wie unterscheiden sie sich von normalen Imports?

© H. Prähofer, 2026

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# Sichtbarkeitsattribute

- **Sichtbarkeitsattribute** erlauben die Sichtbarkeit von Elementen von Klassen einzuschränken
- Sichtbarkeitsattribute unterstützen **Information Hiding**
  - d.h. **inneren Zustand** vor **unerlaubtem Zugriff zu schützen**

konsistenter Zustand kann garantiert werden

## Sichtbarkeitsattribute in Java

| | |
|---|---|
| `private` | Sichtbarkeit auf **Klasse** eingeschränkt |
| *package-private* (kein Attribut) | Sichtbar innerhalb des **gleichen Packages** (nicht in Unterpackages) |
| `protected` | Sichtbar innerhalb des **gleichen Packages** und in den **Unterklassen** (Unterklassen auch aus anderen Packages) |
| `public` | allgemein sichtbar |




© H. Prähofer, 2026

# Timer mit Information Hiding

```java
public class Timer {
  private int hrs;
  private int min;
  private int sec;

  public Timer(int h, int m, int s) {
    hrs = h; min = m; sec = s;
  }
  public void clear() {
    hrs = 0;
    min = 0;
    sec = 0;
  }
  public void add(int sec) {
    this.sec += sec;
    if (this.sec > 59) { min += sec / 60; this.sec = sec % 60; }
    if (min > 59) { hrs += min / 60; this.min = min % 60; }
  }
  @Override
  public String toString() {
    return String.format("%d:%02d:%02d", hrs, min, sec);
  }
}
```

private: Felder
➔ Man kann von außen nicht auf Datenfelder zugreifen

public: Konstruktor und Methoden
**add**, **clear**, **toString**

```java
Timer t = new Timer(0, 1, 30);
t.add(40);
Out.println(t.toString());
t.clear();
```

kein direkter Zugriff auf Felder
➔ interner Zustand abgesichert

```java
t.min = 122;
Out.println(t.hrs);
```

✗ Fehler! Kein Zugriff

© H. Prähofer, 2026

# Getter und Setter

**Felder sollen private sein**

**Zugriffsmethoden zu Feldern**

- *Getter* – Lesen des Feldes
- *Setter* – Schreiben des Feldes

realisieren Eigenschaften (*Properties*)

**Schema*:**

```
public class Classname {

  private Type field;

  public Type getField() {
    return field;
  }

  public void setField(Type field) {
    ...
    this.field = field;
  }
}
```

Feld private

Getter

Setter

Bei Booleschen Properties Getter mit **is** statt **get**

```
public class Classname {

  private boolean field;

  public boolean isField() {
    return field;
  }

  ...
}
```

Feld

Getter

*festgelegt in: Java Beans Specification, https://download.oracle.com/otndocs/jcp/7224-javabeans-1.01-fr-spec-oth-JSpec/

© H. Prähofer, 2026

# Getter und Setter

## Beispiel Timer

```
public class Timer {
  private int hrs, min, sec;

  public int getHrs() {
    return hrs;
  }
  public void setHrs(int hrs) {
    this.hrs = hrs;
  }

  public int getMin() {
    return min;
  }
  public void setMin(int min) {
    ...
    this.min = min;
  }

  public int getSec() {
    return sec;
  }
  ...
}
```

Felder

Getter für **hrs**

Setter für **hrs**

Detto für **min**

Detto für **sec**

Properties von **Timer**:

- **hrs**
- **min**
- **sec**

© H. Prähofer, 2026

# Getter und Setter: Vorteile

- read-only-Zugriff, d.h. nur *Getter,* keine *Setter*

```
public class Timer {
  private int hrs, min, sec;
  public int getHrs() { return hrs; }
  public int getMin() { return min; }
  public int getSec() { return sec; }
  ...
}
```

keine Setter definiert

- Properties können auch berechnet werden, z.B. Berechnung von hrs, min, sec aus Feld **totalSec**

```
public class Timer {
  private int totalSec;
  public int getHrs() { return totalSec / 3600; }
  ...
}
```

Berechnung der Properties **hrs**, **min**, **sec** aus **totalSec**

- Zusatzaktionen beim Zugriff möglich (z.B. Logging, Plausibilitätsprüfungen, ...)

```
public class Timer {
  ...
  public void setMin(int min) {
    if (min < 0 || min >= 60) throw new IllegalArgumentException();
    this.min = min;
  }
}
```

Prüfung vor Setzen

- get-/set-Methoden können in Unterklassen überschrieben werden (Felder nicht)

Siehe Kapitel 2. Vererbung

© H. Prähofer, 2026

# Getter: Alternative Namensgebung

## Name von Getter gleich Feldnamen

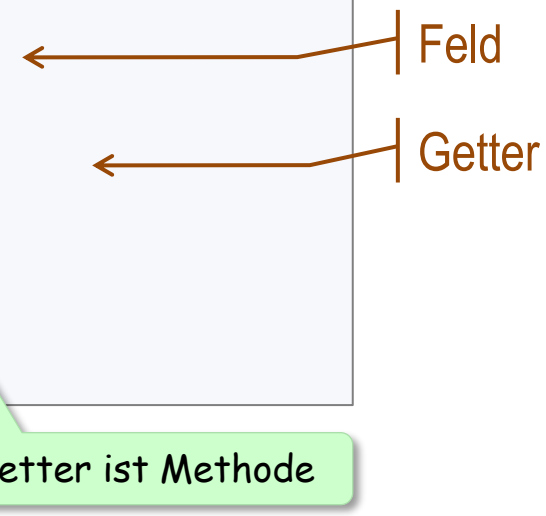
### Schema

```
public class Classname {

  private Type field;

  public Type field() {
    return field;
  }

}
```




### Beispiel Timer

```
public class Timer {
  private int hrs, min, sec;

  public int hrs() {
    return hrs;
  }

  public int min() {
    return min;
  }

  public int sec() {
    return sec;
  }
  ...
}
```

© H. Prähofer, 2026

# Richtlinien zur Verwendung der Sichtbarkeitsattribute

**private**

- Felder von öffentlichen Klassen
- Hilfsmethoden, die nur in Klasse verwendet werden sollen

**package-private**

- Hilfsklassen in einem Package, die für interne Implementierung und nicht zur direkten Verwendung gedacht sind
- Konstruktoren und Methoden von öffentlichen Klassen, die nur innerhalb des Packages verwendet werden sollen

**protected**

- bei gewollter Vererbung (siehe Kapitel Vererbung)

**public**

- nur Klassen und deren Methoden, die zur Verwendung vorgesehen sind

© H. Prähofer, 2026

# Lernziele überprüfen

- Was versteht man unter Information Hiding (Datenkapselung) und warum ist diese wichtig?
- Welche Sichtbarkeitsattribute kennt Java und wie unterscheiden sie sich?
- Welche Richtlinien gelten für den Einsatz von private, package-private, protected und public in Klassen, Methoden und Feldern?
- Was sind Getter- und Setter-Methoden und warum werden diese verwendet?
- Wie ist die Konvention für die Implementierung von Setter- und Getter-Methoden?

© H. Prähofer, 2026

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# final: Statische Felder

**final Variablen sind unveränderbar, d.h., können nur einmal gesetzt werden**

**final statische Felder**

- sind **Programmkonstanten**
- werden mit **Großbuchstaben** geschrieben, mit Underscore als Trennzeichen
- Wert muss **direkt bei Deklaration** oder im **statischen Initialisierungsblock** zugewiesen werden

```
public final class Integer ... {

  public static final int MAX_VALUE = 0x7fffffff;

  ...
}
```

© H. Prähofer, 2026

# final: Objektfelder

## final Objektvariablen

„immutable"

- müssen **im Konstruktor gesetzt** werden (oder direkt bei Deklaration)
- können **dann nicht mehr verändert** werden

Beispiel: Klasse **Fraction** für Bruchzahlen

```
public class Fraction {
    private final int num;
    private final int denom;

    public Fraction(int num, int denom) {
        if (denom == 0) throw new IllegalArgumentException();
        this.num = num;
        this.denom = denom;
    }
    public int num() {
        return num;
    }
    public int denom() {
        return denom;
    }
   ...
}
```

Werte der final Variablen müssen im Konstruktor "fixiert" werden!

nur Getter (Setter nicht möglich)

```
Fraction f1 = new Fraction(1, 2);
Out.println(f1.num() + "/" + f1.denom());
```

1/2

```
Fraction f2 = new Fraction(2, 3);
Out.println(f2.num() + "/" + f2.denom());
```

2/3

© H. Prähofer, 2026

# Unveränderliche Datenklassen

- Objekte mit nur **unveränderlichen** Feldern
- Sollen **equals** und **hashCode** für Wertevergleich implementieren

```
public class Fraction {
  private final int num, denom;

  public Fraction(int num, int denom) {
    if (denom == 0) throw new IllegalArgumentException();
    this.num = num; this.denom = denom;
  }

  public int num() { return num; }
  public int denom() { return denom; }

  @Override
  public boolean equals(Object o) { ...}

  @Override
  public int hashCode() { ... }

  @Override
  public String toString() { return num + "/" + denom; }
}
```

**Vorteile unveränderlicher Objekte:**

- verhindern Seiteneffekte
- besitzen ähnliche Eigenschaften wie Standarddatentypen

© H. Prähofer, 2026

# Implementierung equals und Hashcode

## equals für Wertevergleich

- Vergleich der internen Feldwerte auf Gleichheit

```
public class Fraction {
  private final int num;
  private final int denom;
  ...
  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    Fraction fraction = (Fraction) o;
    return num == fraction.num &&
           denom == fraction.denom;
  }
}
```

### Schema zur Implementierung von equals

**equals** aus **Object** überschreiben
immer mit Parameter von Typ **Object**

**nie** anders

zuerst auf **Referenzgleichheit** prüfen
referenzgleiche Objekte sind immer gleich !

dann Parameter auf **null** und **Klassen vergleichen**
wenn **null** oder Klassen nicht ident dann nicht gleich

dann **Typecast** des Arguments

dann **ausgewählte Felder** vergleichen
für Referenztypen wieder mit **equals**

© H. Prähofer, 2026

# Implementierung equals und Hashcode

## Bedingungen zur Implementierung hashCode

- **equals** und **hashCode** immer zusammen
- immer so, dass Folgendes gilt:

$$a.equals(b) == true \Rightarrow a.hashCode() == b.hashCode()$$

umgekehrt nicht unbedingt notwendig

d.h., sind zwei Objekte **a** und **b equals**, so müssen auch ihre **hashCode**s gleich sein

## Schema zur Implementierung von hashCode

- für **hashCode** am besten gleiche Felder wie bei **equals** verwenden
- am besten **Objects.hash** verwenden

Näheres bei Abschnitt zu Collections

```
import java.util.Objects;

public class Fraction {
  ...
@Override
  public int hashCode() {
    return Objects.hash(num, denom);
  }
}
```

© H. Prähofer, 2026

# Implementierung equals und Hashcode

## Beispiel Person

```
public class Person {
  private final String firstName;
  private final String lastName;
  private final int age;
  ...
  @Override
  public boolean equals(Object o) {
    if (this == o) {
      return true;
    }
    if (o == null || getClass() != o.getClass()) {
      return false;
    }
    Person person = (Person) o;
    return firstName.equals(person.firstName) &&
           lastName.equals(person.lastName) &&
           age == person.age;
  }
  @Override
  public int hashCode() {
    return Objects.hash(firstName, lastName, age);
  }
}
```

Vergleich von Referenztypen mit **equals**

© H. Prähofer, 2026

# Unveränderliche Datenobjekte: Operationen

- Operationen mit unveränderlichen Objekten erzeugen immer neue Objekte

```
public class Fraction {
  private final int num, denom;

  public Fraction(int num, int denom) {
    if (denom == 0) throw new IllegalArgumentException();
    this.num = num; this.denom = denom;
  }
  public int num() { return num; }
  public int denom() { return denom; }

  public Fraction mult(Fraction f) {
    return new Fraction(this.num * f.num, this.denom * f.denom);
  }

  public Fraction add(Fraction f) {
    return new Fraction(this. num * f.denom + f.num * this. denom, this. denom * f.denom);
  }

  ...

}
```

Objekte selbst werden nie verändert

Operationen erzeugen und liefern neue Objekte

© H. Prähofer, 2026

# Veränderliche vs. unveränderliche Klassen: Beispiel Timer und Time

## Timer erlaubt zählen → veränderlich

- Methoden verändern das Objekt
- kein **equals** und **hashCode**

```
public class Timer {
  private int hrs, min, sec;

  public Timer(int hrs, int min, int sec) { ... }

  public void clear() {
    hrs = 0; min = 0; sec = 0;
  }
  public void add(int s) {
    this.sec += s;
    if (sec > 59) {
      min += sec / 60;
      sec = sec % 60;
    }
    if (min > 59) {
      hrs += min / 60;
      min = min % 60;
    }
  }
}
```

Methoden ändern Objekt

## Time stellt Zeitpunkt dar → unveränderlich

- Methoden liefern neue Objekte
- **equals** und **hashCode** implementiert

```
public class Time {
  private final int hrs, min, sec;

  public Time(int hrs, int min, int sec) { ... }

  ...
  @Override
  public boolean equals(Object o) { ... }

  @Override
  public int hashCode() { ... }

  public Time after(int sec) {
    int totalSec = ...;
    return new Time(totalSec / 3600,
                    (totalSec % 3600) / 60,
                    totalSec % 60);
  }

}
```

Methoden erzeugen und liefern neue Objekte

© H. Prähofer, 2026

# Best Practice: Möglichst unveränderliche Klassen

\+ **keine unbeabsichtigten Seiteneffekte** möglich

\+ **keine Datenkonflikte** bei gleichzeitigem und parallelem Zugriff

\+ ermöglichen **Compileroptimierungen**

➔ dadurch **zuverlässige** und **effiziente Programme**

© H. Prähofer, 2026

# Lernziele überprüfen

- Welche Bedeutung hat das Schlüsselwort final bei Variablen in Java?
- Wie unterscheiden sich final Objektvariablen und Klassenvariablen?
- Wie ist die Schreibweise für Variablen von final Klassenvariablen?
- Wann und wie müssen final Objektvariablen initialisiert werden?
- Was bedeutet der Begriff *unveränderlich* (*immutable*) im Zusammenhang mit Klassen und Objekten?
- Welche Rolle spielen equals und hashCode bei unveränderlichen Datenklassen?
- Skizzieren Sie das Schema der Implementierung von equals und hashCode?
- Was sind die Vorteile von unveränderlichen Objekten?

© H. Prähofer, 2026

# 1. Klassen


1.1 Klassen, Objekte und Referenzen

1.2 Bestandteile von Klassen

1.3 Packages

1.4 Sichtbarkeitsattribute und Information Hiding

1.5 final Variablen und unveränderliche Objekte

1.6 Fallstudie zum Klassenentwurf


© H. Prähofer, 2026

# Richtlinien zur Gestaltung von Klassensystemen

## Unterscheidung in Datenklassen und API-Klassen

- **Datenklassen** stellen die Anwendungsdaten dar
- **API-Klassen** die operativen Schnittstellen des Programmsystem

## Datenklassen

- Datenklassen möglichst unveränderlich machen
  - ➔ keine ungewollten Seiteneffekte
- Für Datenklassen ist die Implementierung von **`equals`** und **`hashCode`** wichtig
  - ➔ siehe Kapitel Collections
- Objekterzeugung und Initialisierung
  - vollständige Initialisierung durch Konstruktor
  - öffentliche Konstruktoren legen die Art der Objekterzeugung dauerhaft in der API fest
    - damit verbaut man sich die spätere Abstraktion und Spezialisierung
    - oft ist es flexibler, Objekte über Methoden zu erzeugen

© H. Prähofer, 2026

# Richtlinien zur Gestaltung von Klassensystemen

## API-Klassen

- API-Methoden zur Implementierung der operationalen Schnittstelle
  - **vollständig**: Operationen müssen alle Anforderungen abdecken
  - **elementar**: Operationen vermeiden, die man durch Kombination von elementaren Operationen realisieren kann
  - **einfach und klar**: Die Funktion einer Operation soll sich klar aus Schnittstelle ergeben
    ➔ Namenswahl ist entscheidend

- **Information Hiding**: keine ungewollten Seiteneffekte, die das System korrumpieren könnten
  - jeder Aufruf muss von einem korrekten Zustand zu einem nächsten korrekten Zustand führen
  - Aufrufe müssen abgesichert sein, d.h. es darf nie ein inkorrekter Zustand entstehen können
  - Keine ungewollten Seiteneffekte bei zurückgegebenen Objekten
    - Z.B. Keine direkten Veränderungen von intern verwendeten Datenstrukturen
      ➔ siehe Kapitel Collections
    - Für Rückgabewerte am besten unveränderliche Objekte verwenden

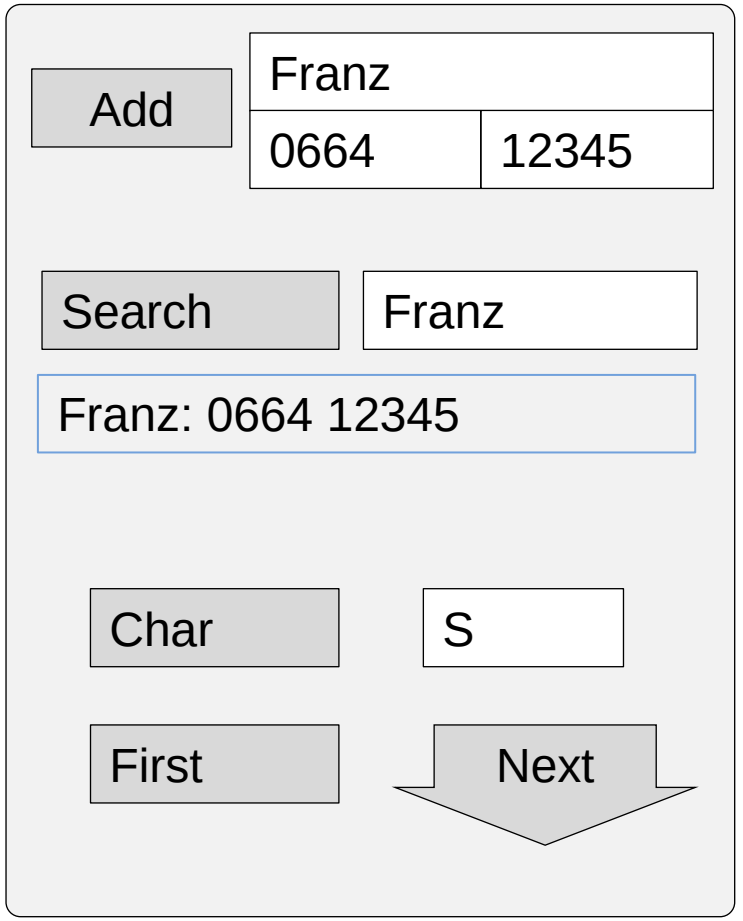
© H. Prähofer, 2026

# Fallbeispiel Telefonbuch

- Einfaches Programmsystem zur Verwaltung von Telefonbucheinträgen
- Einträge mit Namen, Vorwahl und Nummer
- Einträge nach Namen sortiert
- Funktionen:
  - Anfügen von Einträgen
  - Suchen mit Namen
  - Prüfen, ob Eintrag enthalten
  - Blättern von einem zum nächsten
  - Springen zum nächsten Eintrag mit gegebenem Buchstaben

➔ Programmsystem könnte Basis für GUI-Anwendung sein

Entwurf GUI:




© H. Prähofer, 2026

# Fallbeispiel Telefonbuch: Klassen

- **Entry**: Datenklasse für einen Eintrag im Telefonbuch mit Namen, Vorwahl und Nummer

- **PhoneBook**: Klasse mit operationalen Schnittstelle zur Verwaltung der Einträge
  - ☐ Anfügen von Einträgen
  - ☐ Update von Nummer und Name
  - ☐ Suchen mit Namen
  - ☐ Springen zum ersten Eintrag
  - ☐ Springen zum nächsten Eintrag mit gegebenem Buchstaben
  - ☐ Blättern von einem zum nächsten Eintrag

- **EntryList** und **Node**: verkettete Liste mit den Einträgen, Verkettung mit Klasse Node

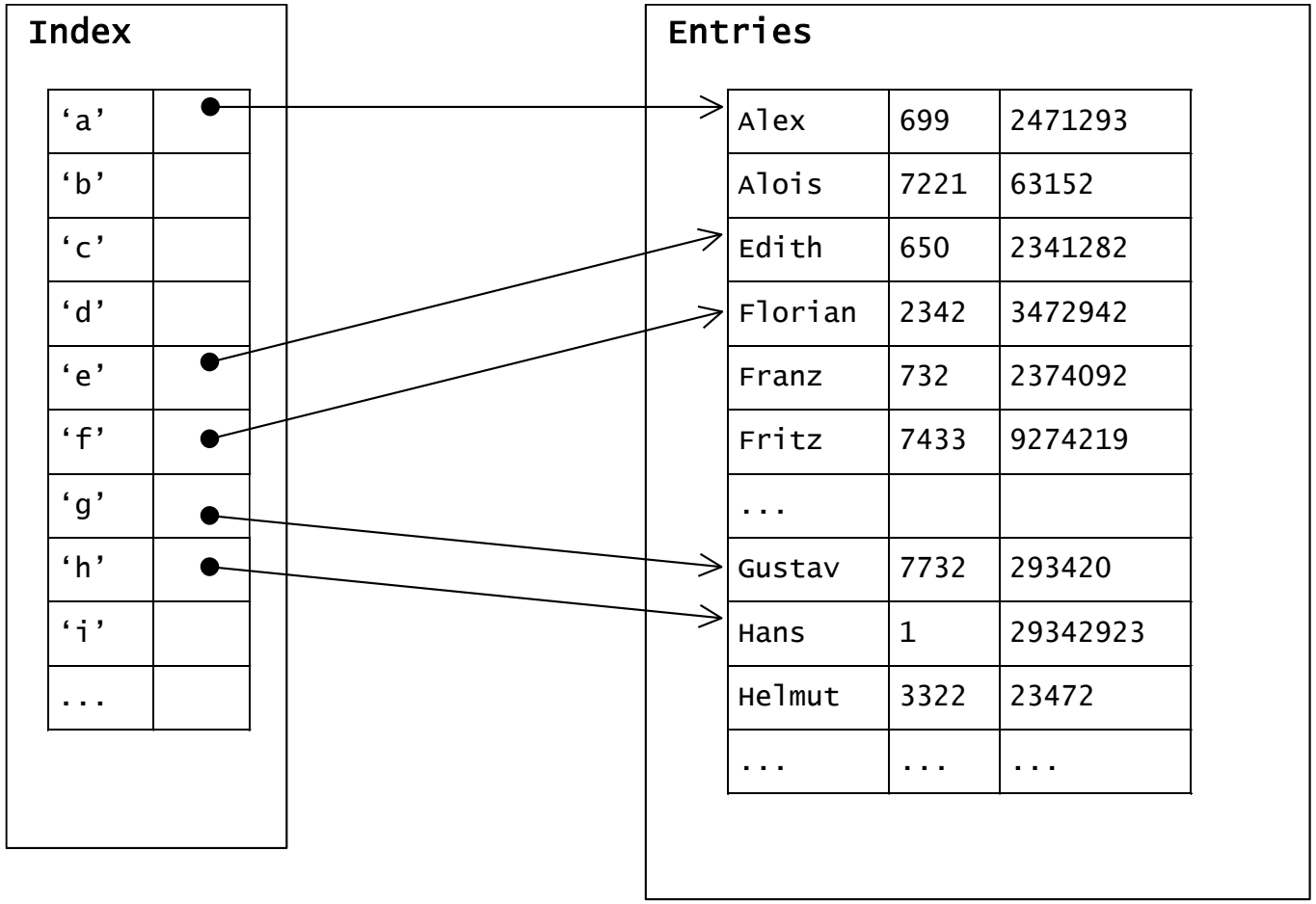
- **PhoneBookIndex**: Indexstruktur von Buchstaben zum ersten Knoten mit oder nach dem Buchstaben

- **PhoneBookApplication**: Konsolenapplikation zur Verwendung des Telefonbuchs

© H. Prähofer, 2026

# Fallbeispiel Telefonbuch: Klassen

- PhoneBookIndex




© H. Prähofer, 2026

# Fallbeispiel Telefonbuch: Aufgaben

## Im Projekt im Download fehlen

- die Sichtbarkeitsattribute bei Klassen, Feldern und Methoden und der Modifier final
- die Methoden der Klasse PhoneBook für die operationale Schnittstelle

## Ihre Aufgabe bis zur nächsten Woche

- Überlegen der
  - ☐ Sichtbarkeitsattribute bei Klassen, Feldern und Methoden
  - ☐ Einsatz von final bei Feldern
- Entwurf der Methoden der operationalen Schnittstelle in Klasse PhoneBook
  - ☐ Namenswahl
  - ☐ Parameter und Rückgabewerte
  - ☐ Exceptions

Nur Methodensignaturen,
keine konkrete Implementierung

© H. Prähofer, 2026